



MLflow: Model Lifecycle Management



1 What is MLflow?

MLflow is an **open-source platform** (created by Databricks) that helps you **manage the entire ML lifecycle** — from **experiment tracking** → **model storage** → **deployment** → **registry**.

You can think of it as a “logbook + storage system” for all your ML work.

It makes sure you can always answer:

“Which data, code, and parameters produced this model?”



2 Core Components of MLflow

Component	Description	Example
1. MLflow Tracking	Logs experiments (parameters, metrics, code, and output)	<code>mlflow.log_metric("accuracy", 0.94)</code>
2. MLflow Projects	Packages code into reproducible runs	project with <code>MLproject</code> file
3. MLflow Models	Defines how to package models for serving	<code>mlflow.sklearn.log_model(model, "model")</code>
4. MLflow Registry	Central store for versioned models	<code>mlflow.register_model("runs:/abc/model", "MyModel")</code>

Together, they cover the whole **MLOps lifecycle**.



3 MLflow Lifecycle Overview

1. Start experiment
2. Train model (log params, metrics)

3. Save model to MLflow
4. Compare runs (choose best)
5. Register in Model Registry
6. Deploy (locally or cloud)

Every experiment → stored with a **Run ID**, **User**, **Date**, and **Results**.

4 MLflow Tracking Explained

➤ Purpose:

Tracks everything you do in a machine learning experiment:

- Parameters → hyperparameters
- Metrics → accuracy, loss, precision
- Artifacts → trained model, plots, CSVs
- Source code → which file generated it

This helps you compare experiments later.

➤ Typical folder structure:

```
mlruns/  
├── 0/  
│   ├── meta.yaml  
│   ├── metrics/  
│   ├── params/  
│   └── artifacts/
```

5 Tracking URI

The **Tracking URI** tells MLflow *where* to log your data.

Default: it stores locally under `/mlruns`.

You can change it to a **remote tracking server** (useful when multiple teammates share results).

```
mlflow.set_tracking_uri("http://127.0.0.1:5000")
```

Examples:

- `file:///home/user/mlruns` → local logs
- `http://mlflow-server:5000` → remote server
- `databricks` → Databricks workspace

6 Creating and Starting Runs

You can use `start_run()` to begin a logging session.

```
import mlflow
```

```
with mlflow.start_run():  
    mlflow.log_param("learning_rate", 0.01)  
    mlflow.log_metric("accuracy", 0.92)
```

Explanation:

- `start_run()` → begins experiment run
 - `log_param()` → saves hyperparameter
 - `log_metric()` → saves performance result
 - MLflow auto-saves timestamps and Run IDs
-

7 Logging Parameters, Metrics, and Artifacts

Function	Purpose	Example
<code>mlflow.log_param()</code>	Logs single parameter	<code>mlflow.log_param("lr", 0.01)</code>
<code>mlflow.log_params()</code>	Logs multiple parameters	<code>mlflow.log_params({"n_estimators": 100, "max_depth": 5})</code>
<code>mlflow.log_metric()</code>	Logs a single metric	<code>mlflow.log_metric("accuracy", 0.95)</code>
<code>mlflow.log_metrics()</code>	Logs multiple metrics	<code>mlflow.log_metrics({"precision": 0.92, "recall": 0.88})</code>
<code>mlflow.log_artifact()</code>	Logs file (plots, model.pkl, etc.)	<code>mlflow.log_artifact("plot.png")</code>

8 Logging Models

MLflow supports all major frameworks: sklearn, TensorFlow, PyTorch, XGBoost, LightGBM, etc.

Example (for sklearn):

```
mlflow.sklearn.log_model(model, "model")
```

You can later **load** it back:

```
model = mlflow.sklearn.load_model("runs:./<RUN_ID>/model")
```

This allows model reproducibility across machines.

9 MLflow Model Registry

Once you have trained multiple models, you can **register** the best one in a central **Model Registry**.

```
from mlflow.tracking import MlflowClient
```

```
client = MlflowClient()
client.create_registered_model("MyBestModel")
client.create_model_version("MyBestModel", "runs:/12345/model", "Initial version")
```

States:

- **Staging** → testing phase
- **Production** → deployed
- **Archived** → old version

10 Why MLflow Matters in MLOps

Benefit	Description
Reproducibility	Anyone can rerun the same experiment
Collaboration	Teams can compare results
Traceability	Full record of parameters & code used
Automation	Integrates with CI/CD & cloud services
Compatibility	Works with AWS, Azure, GCP easily

In short, it is the “**glue**” connecting Data Science and MLOps pipelines.

Example Code: Mini MLflow Experiment

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
```

```
from sklearn.model_selection import train_test_split
```

```
# 1. Set tracking server
```

```
mlflow.set_tracking_uri("http://127.0.0.1:5000")
```

```
mlflow.set_experiment("iris-classification")
```

```
# 2. Prepare data
```

```
X, y = load_iris(return_X_y=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

```
# 3. Start run
```

```
with mlflow.start_run():
```

```
    clf = RandomForestClassifier(n_estimators=100)
```

```
    clf.fit(X_train, y_train)
```

```
    acc = clf.score(X_test, y_test)
```

```
    mlflow.log_param("n_estimators", 100)
```

```
    mlflow.log_metric("accuracy", acc)
```

```
    mlflow.sklearn.log_model(clf, "model")
```

15 MCQs (with answers)

- 1 MLflow was created by → **Databricks**
 - 2 The command `mlflow.set_tracking_uri()` defines → **Where to log experiments**
 - 3 `mlflow.start_run()` → **Begins an experiment session**
 - 4 To record parameters, we use → `mlflow.log_param()`
 - 5 To log accuracy, we use → `mlflow.log_metric("accuracy", value)`
 - 6 To log a file (plot or model), use → `mlflow.log_artifact()`
 - 7 Default local storage path for runs → `mlruns/` folder
 - 8 MLflow supports → **TensorFlow, PyTorch, XGBoost, sklearn, etc.**
 - 9 Which MLflow module handles reproducible code packages? → **Projects**
 - 10 Which component stores versioned models? → **Model Registry**
 - 11 The 3 main states of a registered model → **Staging, Production, Archived**
 - 12 MLflow Tracking logs → **Params, metrics, code, and artifacts**
 - 13 Benefit of using Tracking URI → **Centralized experiment tracking**
 - 14 MLflow integrates with → **All major clouds (AWS, Azure, GCP)**
 - 15 Purpose of MLflow → **Track, compare, and manage ML experiments**
-



10 Short Answer Questions

1 What is MLflow used for?

It's a platform to track, log, store, and manage all parts of an ML experiment from training to deployment.

2 What is a Tracking URI?

A link or path that tells MLflow where to save logs (local folder, remote server, or Databricks).

3 What does `mlflow.start_run()` do?

It starts a logging session for a specific model training experiment.

4 Difference between `log_param` and `log_metric`.

`log_param` saves input parameters (like learning rate), `log_metric` saves performance values (like accuracy).

5 What is logged as an artifact?

Files such as trained models, charts, confusion matrices, or CSVs.

6 What are the benefits of MLflow Registry?

Centralized model versioning, lifecycle states (Staging/Production), and easy rollbacks.

7 What happens when you use MLflow with CI/CD?

Each pipeline run automatically logs metrics, allowing automatic retraining and promotion.

8 Can MLflow be used with other frameworks?

Yes — it's framework-agnostic and works with scikit-learn, PyTorch, TensorFlow, and even custom models.

9 What's the difference between local and remote tracking?

Local = only on your machine (`mlruns/`), Remote = shared tracking server for teams.

10 How does MLflow improve reproducibility?

By storing all experiment info (code, params, data path, metrics) so any team member can exactly rerun it later.